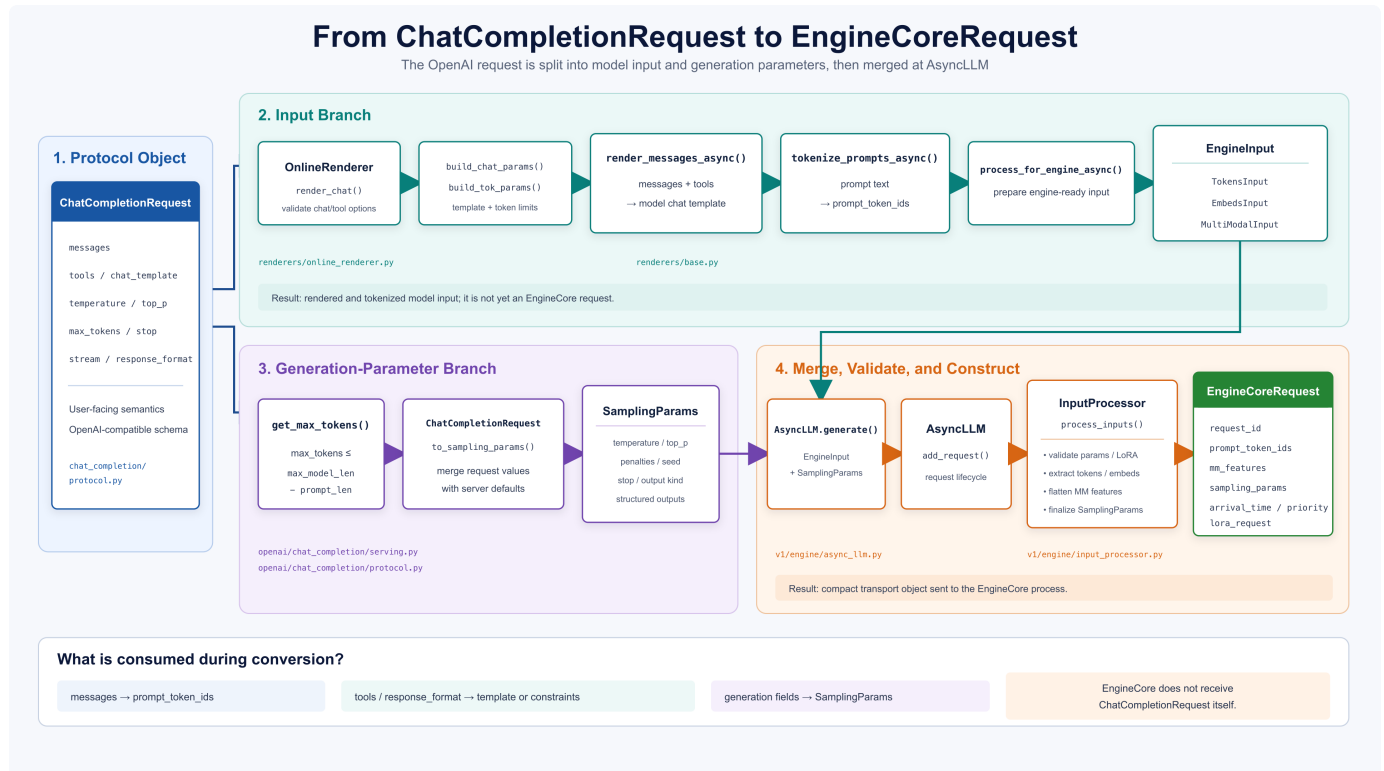


第5课 请求预处理：Chat Template 与 Tokenization



1. Protocol Object

HTTP 入口是:

POST /v1/chat/completions

```
1  async def create_chat_completion(
2      request: ChatCompletionRequest,
3      raw_request: Request,
4  ):
5      handler = chat(raw_request)
6      generator = await handler.create_chat_completion(request, raw_request)
```

FastAPI/Pydantic 将 JSON 请求体转换成:

ChatCompletionRequest

它同时包含两类信息：

```
1  模型输入：  
2  messages、tools、chat_template、多模态内容  
3  
4  生成控制：  
5  temperature、top_p、max_tokens、stop、response_format
```

这也是后面需要拆成两条分支的原因。

2. Input Branch

2.1 进入 OpenAIServingChat

主要入口是：

[OpenAIServingChat._create_chat_completion\(\)](#)

这里首先调用：

```
1  result = await self.render_chat_request(request)
```

它经过模型检查和 Engine 健康检查后，委托给：

```
1  return await self.online_renderer.render_chat(request)
```

对应代码：

[render_chat_request\(\)](#)

2.2 OnlineRenderer 做 API 语义验证

[OnlineRenderer.render_chat\(\)](#) 负责：

- 验证 `tool_choice`

- 检查工具解析器是否可用
- 决定是否把 `tools` 传给 Chat Template
- 验证客户端是否允许覆盖 `chat_template`
- 处理 Mistral 和 GPT-OSS 等特殊路径

普通 Hugging Face 模型最后进入：

```
1 conversation, engine_inputs = await self.preprocess_chat(...)
```

2.3 构造 ChatParams 和 TokenizeParams

实现位于：

[OnlineRenderer.preprocess_chat\(\)](#)

```
1 tok_params = request.build_tok_params(self.model_config)
2
3 chat_params = request.build_chat_params(
4     default_template,
5     default_template_content_format,
6 ).with_defaults(...)
```

这两个对象职责不同：

```
1 ChatParams
2     控制消息解析和 Chat Template
3     chat_template、tools、documents、add_generation_prompt
4
5 TokenizeParams
6     控制分词和长度限制
7     max_model_len、max_output_tokens、truncate_prompt_tokens
```

构造代码：

- [build_chat_params\(\)](#)
- [build_tok_params\(\)](#)

2.4 BaseRenderer 执行四阶段转换

接下来调用：

```
1  await renderer.render_chat_async(  
2      [messages],  
3      chat_params,  
4      tok_params,  
5  )
```

核心编排位于：

[BaseRenderer.render_chat_async\(\)](#)

它包含四步：

```
1  # 1. messages → DictPrompt  
2  render_messages_async()  
3  
4  # 2. DictPrompt → TokPrompt  
5  tokenize_prompts_async()  
6  
7  # 3. 添加 cache_salt、mm_processor_kwargs  
8  # cache_salt 影响 prefix cache 的匹配范围  
9  # mm_processor_kwargs 传给多模态 processor 的额外参数  
10 _apply_prompt_extras()  
11  
12 # 4. TokPrompt → EngineInput  
13 process_for_engine_async()
```

2.5 应用模型 Chat Template

普通 HF 模型使用：

[HfRenderer.render_messages_async\(\)](#)

它先把 OpenAI 消息规范化：



Plain Text |

```
1 conversation, mm_data, mm_uuids = (  
2     await parse_chat_messages_async(...)  
3 )
```

然后套用模型 Chat Template:



Plain Text |

```
1 prompt_raw = await self._apply_chat_template_async(  
2     model_config,  
3     tokenizer,  
4     conversation,  
5     **chat_template_kwargs,  
6 )
```

真正调用 Hugging Face 的位置是:

[safe_apply_chat_template\(\)](#)



Plain Text |

```
1 plain = tokenizer.apply_chat_template(  
2     conversation=conversation,  
3     tools=tools,  
4     chat_template=chat_template,  
5     tokenize=tokenize,  
6 )
```

普通 HF 路径通常先返回渲染文本，下一步再统一 tokenize；Mistral、Prompt Embeds 等路径可能在模板阶段直接产生 token IDs。

示例

```
1 conversation = [  
2   {  
3     "role": "system",  
4     "content": "你是一个简洁的助手。",  
5   },  
6   {  
7     "role": "user",  
8     "content": "介绍一下北京。",  
9   }  
10 ]  
11  
12 <|im_start|>system  
13 你是一个简洁的助手。<|im_end|>  
14 <|im_start|>user  
15 介绍一下北京。<|im_end|>  
16 <|im_start|>assistant
```

2.6 分词并转换成 EngineInput

统一分词入口：

`tokenize_prompts_async()`

示例

```
1 dict_prompt = {
2     "prompt": (
3         "<|im_start|>system\n"
4         "你是一个简洁的助手。<|im_end|>\n"
5         "<|im_start|>user\n"
6         "介绍一下北京。<|im_end|>\n"
7         "<|im_start|>assistant\n"
8     )
9 }
10
11 tok_prompt = {
12     "prompt": (
13         "<|im_start|>system\n"
14         "你是一个简洁的助手。<|im_end|>\n"
15         "<|im_start|>user\n"
16         "介绍一下北京。<|im_end|>\n"
17         "<|im_start|>assistant\n"
18     ),
19     "prompt_token_ids": [
20         151644,
21         8948,
22         198,
23         108386,
24         104455,
25         100165,
26         1773,
27         151645,
28         198,
29         151644,
30         872,
31         198,
32         104198,
33         102104,
34         105195,
35         1773,
36         151645,
37         198,
38         151644,
39         77091,
40         198,
41     ],
42 }
```

然后:

`process_for_engine_async()` 把前端使用的 `TokPrompt` 转成推理引擎可以直接接收的标准 `EngineInput`

根据输入类型产生：

- `TokensInput`
- `EmbedsInput`
- `MultiModalInput`
- `EncoderDecoderInput`

这些类型定义在：

[vllm/inputs/engine.py](#)

最终统一为：

`EngineInput` 类型别名

注意：`EngineInput` 不是一个具体 class，而是一组 `TypedDict` 的联合类型。

3. Generation-Parameter Branch

输入分支产生 `EngineInput` 后，服务层已经能够得到真实的 prompt 长度：

```
▼ Plain Text |
1 prompt_token_ids = self._extract_prompt_components(engine_input).token_ids
```

然后计算允许生成的 token 数：

[get_max_tokens\(\)](#)

核心约束是：

```
▼ Plain Text |
1 effective_max_tokens = min(
2     model_max_tokens,
3     request_or_default_max_tokens,
4     override_max_tokens,
5     platform_max_tokens,
6 )
```

接着调用：

`ChatCompletionRequest.to_sampling_params()`

把 OpenAI API 字段转换为内部 `SamplingParams` :

```
1 temperature
2 top_p / top_k / min_p
3 frequency_penalty
4 presence_penalty
5 repetition_penalty
6 stop / stop_token_ids
7 seed
8 logprobs
9 structured_outputs
10 max_tokens
```

因此:

```
1 ChatCompletionRequest
2     | messages      → EngineInput
3     | generation字段 → SamplingParams
```

4. Merge, Validate and Construct

4.1 两条分支在 `generate()` 汇合

服务层调用:

```
1 generator = self.engine_client.generate(
2     engine_input,
3     sampling_params,
4     sub_request_id,
5     ...
6 )
```

代码位置:

[serving.py:357](#)

这里的 `engine_client` 对在线服务而言就是 `AsyncLLM`。

4.2 AsyncLLM.generate()

[AsyncLLM.generate\(\)](#) 做两件核心事情：

```
1  q = await self.add_request(  
2      request_id,  
3      prompt,  
4      sampling_params,  
5      ...  
6  )
```

然后不断从请求的 `RequestOutputCollector` 读取输出并 `yield`：

```
1  while not finished:  
2      out = q.get_nowait() or await q.get()  
3      yield out
```

`generate()` 本身是一个异步生成器，它同时承担请求提交与结果流式返回。

4.3 InputProcessor 构造 EngineCoreRequest

在：

[AsyncLLM.add_request\(\)](#)

调用：

```
1  request = self.input_processor.process_inputs(  
2      request_id,  
3      prompt,  
4      params,  
5      ...  
6  )
```

核心实现：

`InputProcessor.process_inputs()`

它负责：

1. 校验 `SamplingParams` 与模型能力。
2. 校验 LoRA。
3. 校验 Data Parallel rank。
4. 从 `EngineInput` 提取 token IDs 或 embeddings。
5. 拆分 encoder/decoder 输入。
6. 克隆并补全 `SamplingParams`。
7. 将多模态字典展平并排序成 `mm_features`。
8. 构造 `EngineCoreRequest`。

最终代码：

```
1  return EngineCoreRequest(  
2      request_id=request_id,  
3      prompt_token_ids=prompt_token_ids,  
4      prompt_embeds=prompt_embeds,  
5      mm_features=mm_features,  
6      sampling_params=sampling_params,  
7      arrival_time=arrival_time,  
8      lora_request=lora_request,  
9      priority=priority,  
10     ...  
11 )
```

4.4 EngineCoreRequest 是进程间传输对象

定义位于：

[EngineCoreRequest](#)

它使用：

```
▼ Plain Text |
1 class EngineCoreRequest(
2     msgspec.Struct,
3     array_like=True,
4     omit_defaults=True,
5     gc=False,
6 ):

```

这是一个紧凑的序列化结构，主要字段包括：

```
▼ Plain Text |
1 request_id
2 prompt_token_ids / prompt_embeds
3 mm_features
4 sampling_params / pooling_params
5 arrival_time
6 lora_request
7 priority
8 data_parallel_rank
9 trace_headers

```

它已经不再包含完整的 `ChatCompletionRequest`，例如 HTTP 语义、响应格式等前端字段已经被消费或转换。

4.5 前端登记输出状态，并发送给 EngineCore

最终在：

[AsyncLLM._add_request\(\)](#)

```
▼ Plain Text |
1 # 当前 API Server 进程保存输出处理状态
2 self.output_processor.add_request(...)
3
4 # 发送到独立 EngineCore 进程
5 await self.engine_core.add_request_async(request)

```

所以同一个请求在这里分成两个方向：

```
1 EngineCoreRequest
2   └─ OutputProcessor
3       保存前端输出、detokenization 和流状态
4
5   └─ EngineCoreClient
6       跨进程发送给 EngineCore
7       → Request
8       → Scheduler waiting queue
```